



DC – AI – Routing – Switching – Security – Wireless – Automation

ESI-LAG Made Easier with EZ-LAG

An HPE Juniper Networking Techpost

Ridha Hamidi

2023-10-10

HPE



Contents

1	ESI-LAG Made Easier with EZ-LAG	4
1.1	Introduction	4
1.2	Topology	4
1.3	How Does EZ-LAG Work?	5
1.4	Basic EZ-LAG Configuration	18
1.4.1	Peer 1 Configuration	18
1.4.2	Peer 2 Configuration	19
1.4.3	Host-1 Configuration Files (Centos 7)	19
1.4.4	Host-2 Configuration (Ubuntu 20.04)	20
1.5	Verifications	21
1.6	Conclusion	29
1.7	Useful links	29
1.8	Glossary	30
1.9	Acknowledgments	30

List of Figures

1	Reference Fabric Topology	4
2	Reference Topology for EZ-LAG Configuration	5

List of Tables

1	Parameters in Services EVPN Configuration Hierarchy	8
2	Default and Auto-Derived Parameters	14

1 ESI-LAG Made Easier with EZ-LAG

Ridha Hamidi - 10/10/2023

A detailed configuration example that shows how to dual-home data center servers to Juniper leaf switches by using EZ-LAG, a simplified version on ESI-LAG made for customers looking for a smooth transition from Multi-Chassis LAG without having to immediately learn all the features and complexities of EVPN-VXLAN technology.

1.1 Introduction

Dual homing is an essential feature to provide connection redundancy for data center servers. Vendors have implemented multiple solutions to avoid single switch failure in data center fabrics, and to fulfill the need for redundant server connectivity, like Virtual-Chassis and MC-LAG/MLAG. These solutions are generally based on proprietary protocols and have variable levels of complexity for configuration and troubleshooting. In most recent years, and with the increasing adoption of EVPN-VXLAN as the founding technology for building modern and resilient data centers, ESI-LAG became the de-facto dual-homing solution between servers and Top-Of-Rack (TOR) switches. However, customers who are only looking for a better alternative to MC-LAG, adopting EVPN-VXLAN technology solely to fulfill the need for server dual-homing, is a steep transition and a hard decision to make, given the learning curve for this new technology.

1.2 Topology

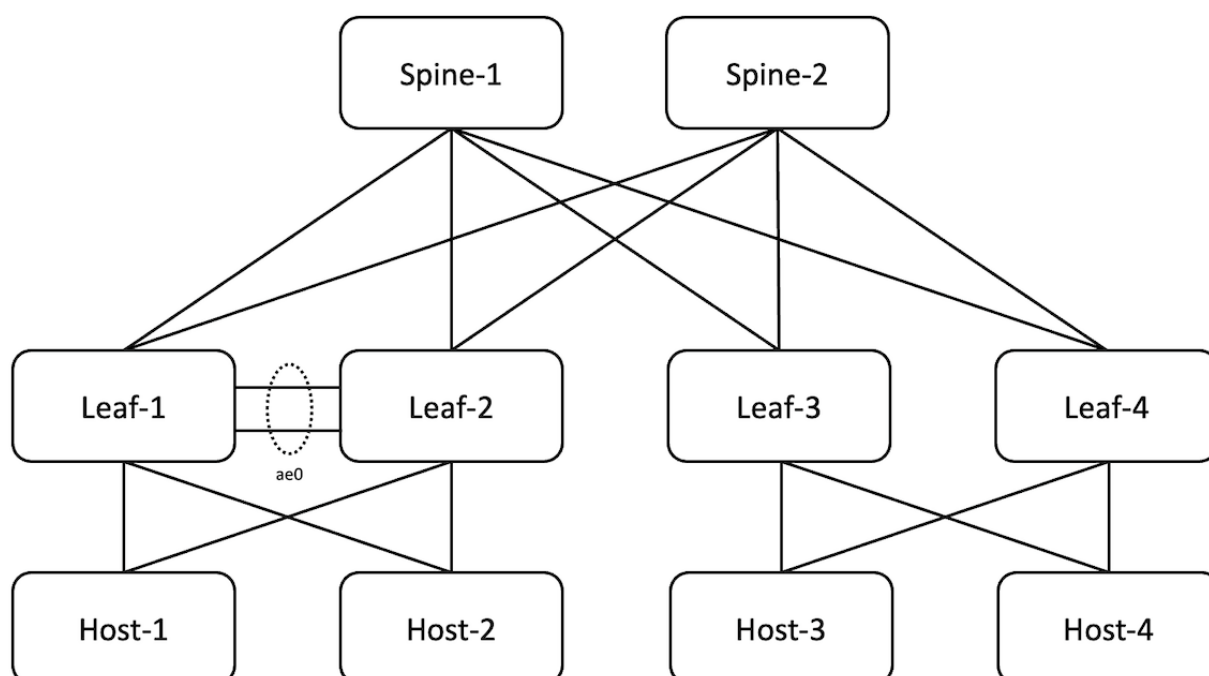


Figure 1: Reference Fabric Topology

- The fabric is preconfigured to provide basic underlay connectivity and all leafs and spines redistribute and receive all lookback addresses via the underlay protocol.

- In this post, we will use EBGP as the underlay protocol, but we will explain later that OSPF can be equally used.
- For the sake of comparing EZ-LAG to ESI-LAG and providing connectivity between all the hosts in the topology, the fabric is configured with BGP as an overlay protocol for family evpn, but only Leaf-3 and Leaf-4 are configured for EVPN-VXLAN. Therefore, Host-3 and Host-4 are dual homed to Leaf-3 and Leaf-4 with ESI-LAG and have overlay connectivity, but there is no connectivity between Host-1/Host-2 and Host-3/Host-4. The objective is to show how to achieve that connectivity by configuring EZ-LAG. Therefore, in the rest of this post, we will focus solely on the configuration of Leaf-1 and Leaf-2 for EZ-LAG.
- In this post, we will use EBGP as the overlay protocol, but we will explain later that IBGP can be equally used.

1.3 How Does EZ-LAG Work?

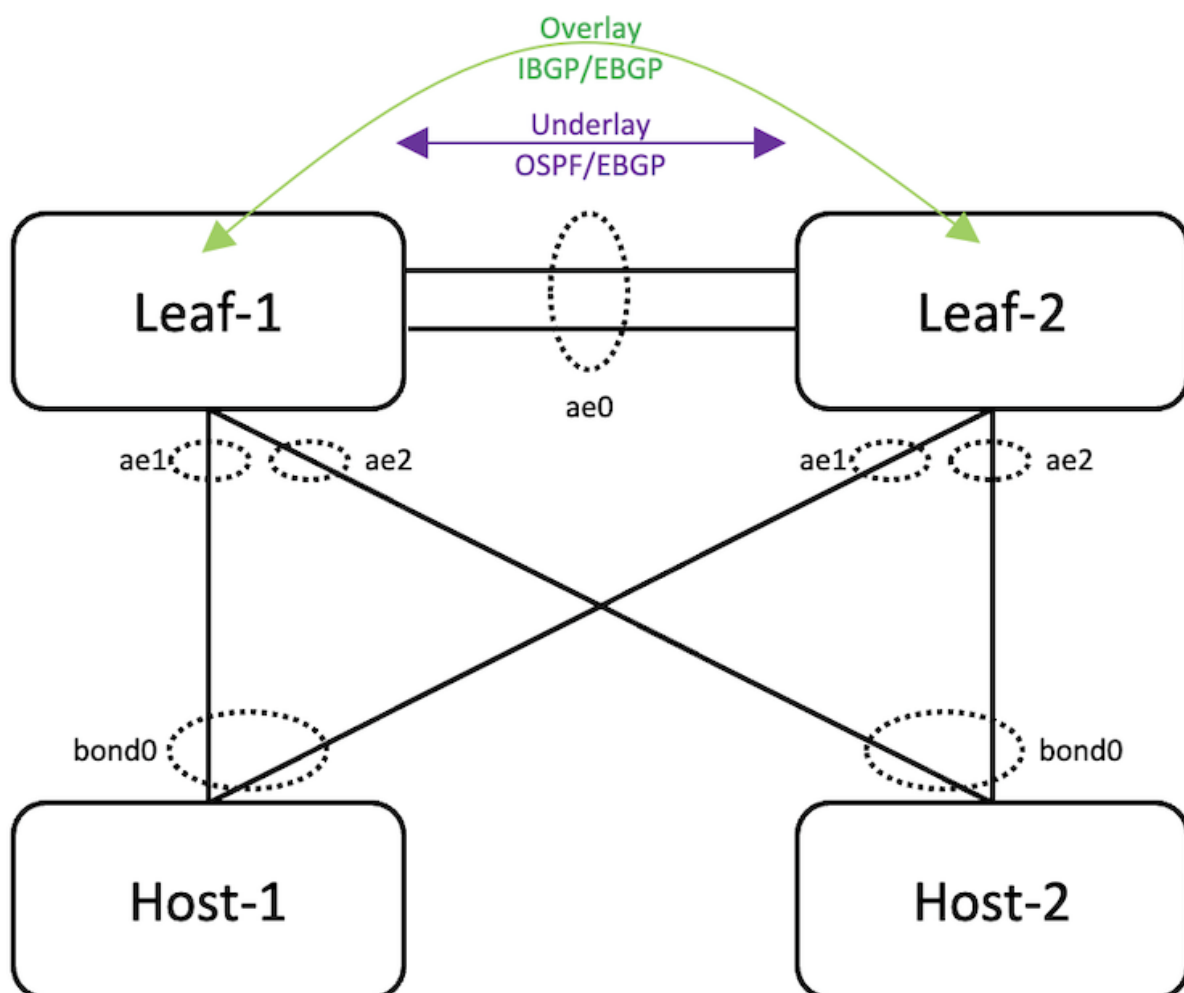


Figure 2: Reference Topology for EZ-LAG Configuration

- A new configuration hierarchy, called “evpn” under “services” stanza has been introduced specifically for EZ-LAG, where the user can specify a limited set of parameters to configure the feature.

- A Python script that the user can call via a commit script. The commit script takes the set of parameters specified by the user and derives all the necessary configuration to build the EZ-LAG. All configuration generated by the commit script will not be visible to the user, unless they explicitly specify “display commit-script”, that’s how the complexity of configuration is hidden from users who want to use the benefits of EZ-LAG feature, without carrying the complexity of its configuration.

```

1  show services evpn {
2      defaults-override {
3          no-aggregate-device-count-config;
4          no-loop-detect-config;
5          no-platform-defaults-config;
6          no-policy-and-routing-options-config;
7          no-storm-control-config;
8          no-underlay-config; # hidden
9      }
10     global-parameters {
11         start-aggregate-ethernet-index [ae-index];
12         default-vlan-id [vlan-id];
13         anycast-mac [mac-address];
14         virtual-gateway {
15             v4-mac [mac-address];
16             v6-mac [mac-address];
17         }
18         virtual-gateway-mac [mac-address];
19         no-irb-address-auto-derive;
20     }
21     device-attribute {
22         peer-id [peer-id];
23         loopback-subnet [subnet-address];
24         system-id [system-id];
25         peer-to-peer {
26             peer-subnet {
27                 inet [ipv4-address];
28                 inet6 [ipv6-address];
29                 interface-name [interface-list];
30             }
31             overlay-connectivity {
32                 ibgp;
33             }
34             underlay-connectivity {
35                 ospf;
36             }
37         }
38     }
39     evpn-vxlan {
40         dhcp-relay [relay-group] {
41             relay-source [relay-source];
42             dhcp-server-address [dhcp-server-address];
43             vrf-instance [instance-id];
44         }
45         irb [irb-instance-name]
46         use-anycast-address;
47         subnet-address {
48             inet [ipv4-subnet-address];
49             inet6 [ipv6-subnet-address];
50             virtual-gateway-v4-address [gateway-v4-address];
51             virtual-gateway-v6-address [gateway-v6-address];
52             no-irb-address-auto-derive;
53         }
54         no-dhcp-relay;
55         vlan-id [vlan-id];
56         instance {
57             vrf-instance [instance-id];
58             mac-vrf-instance [instance-id];
59         }
60         apply-config-groups [group-name];
61     }
62     server [server-name] {
63         esi-lag-id [esi-lag-id];
64         single-home-id [single-home-id];
65         enable-pxe-boot;
66         vlan-id-list [vlan-id-list];
67         mac-vrf-instance [instance-id] vlan-id-list [vlan-id-list];
68         interface [interface-name];
69         apply-config-groups [group-name];
70         no-lacp;
71     }

```

```

72     }
73   }
74   traceoptions {
75     level {
76       all;
77       error;
78       info;
79       notice;
80       verbose;
81       warning;
82     }
83     flag {
84       all;
85       chassis;
86       forwarding-options;
87       input-configuration;
88       Interfaces;
89       policy-options;
90       Protocols;
91       routing-instance;
92       routing-options;
93     }
94   }

```

- “defaults-override” : all configuration knobs under this section are optional ; in this section, the user can specify all parameters they want to preserve, so the commit script will not override them. For example, if the user prefers to use a pre-configured ASN and specify “no-policy-and-routing-options-config” then they will need to configure everything under “routing-options” and “policy-options” hierarchies that are needed for EZ-LAG to function, like a Router-ID, a policy to export loopback addresses in the underlay, and a policy to load balance traffic exported to the forwarding table. Otherwise, an error will be thrown if you try to commit.
- “global-parameters” : all configuration knobs under this section are optional ; in this section, the user specifies the parameters that are common to both peers
- “device-attribute” : all configuration knobs under this section are mandatory, except “overlay-connectivity” ; this section contains parameters that are specific to the local peer. Parameters in this section must be carefully selected because must either match or be compatible with what is configured on the remote peer.
- “evpn-vxlan” : this section contains a mix of mandatory and optional parameters ; in this section, the user specifies the parameters needed to configure EVPN-VXLAN for EZ-LAG to function. This section requires a minimum understanding of how this feature works.

Table 1 below provides more details about the new configuration stanzas, attributes, their purpose, and if they are mandatory or optional:

Table 1: Parameters in Services EVPN Configuration Hierarchy

Object	Attribute	Description	Mandatory
default-override		All configuration parameters in this section are optional. In this section, the user specifies which configuration sections they want to configure manually and to prevent them from being over-written by auto-derived parameters.	No
	no-platform-defaults-config	Do not auto-derive platform specific configurations.	No
	no-aggregate-device-count-config	EZ-LAG needs at least two ae interfaces to function, ae0 for inter-chassis link, then one per dual-homed server. Device count must be configured otherwise the script will use the platform default, commonly 255	No

Object	Attribute	Description	Mandatory
global-parameters	no-policy-and-routing-options-config	EZ-LAG script needs to configure autonomous-system and router-id under routing-options section, and per-packet load-balancing and export loopback under policy-options section. This knob id to allow the user to configure these manually.Note that if policies are configured manually, their names must be “pplb” and “EXPORT-LO0”	No
	no-loop-detect-config	This knob can be used to configure “protocols loop-detect” section manually. The auto-derived configuration of this section will be shown later in this post	No
	no-storm-control-config	This know lets the user configure storm-control manually under interfaces and forwarding-options sections. More details on this later in this post.	No
	no-underlay-config	This is a hidden knob that instructs the script not to generate an underlay configuration section.At the time of writing this post, this knob is not implemented	No
		This section contains parameters that are common across peers	

Object	Attribute	Description	Mandatory
virtual-gateway	start-aggregate-ethernet-index [ae-index]	By default, the script starts assigning dual-homed server interfaces starting from ae1. This knob allows to start the numbering from a different index to avoid potential conflicts	No
	default-vlan-id [vlan-id]	Specify the default VLAN ID. The default is system-default, generally 0	No
	anycast-mac [mac-address]	In case anycast is used for irb interfaces, this knob allows to configure the anycast MAC address. If virtual-gateway is used, this MAC address will be used as virtual-gateway-v4-mac	No
	no-irb-address-auto-derive	Do not auto-derive address and use the subnet address as is	No
		You can configure either virtual-gateway-mac or virtual-gateway and then either v4-mac or v6-mac or both parameters	No
	v4-mac [mac-address]	The default is 00:00:5e:00:01:01	No
virtual-gateway-mac	v6-mac [mac-address]	The default is 00:00:5e:00:02:01	No
	[mac-address]	This applies both to v4 and v6 gateways	
device-attribute		This section contains parameters that are specific to local peer device	

Object	Attribute	Description	Mandatory
peer-to-peer	peer-id [peer-id]	Local peer ID. It's important to choose this parameter carefully for both peers, because many other parameters are auto-derived by using this peer-id. One of the peers must have peer-id=1 and the other must be different than 1	Yes
	loopback-subnet [loopback-subnet]	lo0 prefix for deriving loopback address	Yes
	system-id [system-id]	System identifier in MAC address format ; this parameter must be identical on both peers, and will be used to auto-generate one system-id per ae interface, by incrementing by esi-lag-id for each ae interface.	Yes
	peer-subnet [inet]	IPv4 address to be used for ae0 interface. Must be /31 subnet. NOTE: either ipv4 or ipv6 is mandatory	Yes
	peer-subnet [inet6]	IPv6 address to be used for ae0 interface. Must be a /127 subnet. NOTE: either ipv4 or ipv6 is mandatory	Yes
	peer-subnet interface [interface-name]	Peer to peer interface or list of interfaces between "[]" brackets	Yes
	overlay-connectivity ibgp	Option to choose IBGP for the overlay. NOTE: default is EBGP	No

Object	Attribute	Description	Mandatory
evpn-vxlan	underlay-connectivityospf	Option to choose OSPF for the underlayNOTE: default is EBGp	No
		Parameters for routing-instance and ESI-LAG configuration	
	dhcp-relay	Dynamic Host Configuration Protocol relay configuration	
group-name		Group name to identify a Server/Relay group	No
irb-instance	relay-source	Interface name for relay source	No
	dhcp-server-address [ip-address]	IP Address of one or more DHCP servers	No
	vrf-instance	Identifier for VRF instance (1..255)	No
		Attribute specific to l3-interface/irb/etc	Yes
	use-anycast-address	Option to use anycast address instead of an irb gateway.If you configure this knob, you must also configure the anycast-mac address	No
	interface-name	Name of irb interface ; there could be multiple irb interfaces.	Yes
	subnet-address inet [ipv4-subnet-address]	Ipv4 address for irb gatewayNOTE: Either inet or inet6 is mandatory	See Note
	subnet-address inet6 [ipv6-subnet-address]	Ipv6 address for irb gatewayNOTE: Either inet or inet6 is mandatory	See Note
	subnet-address no-irb-address-auto-derive	Do not auto-derive address and use the subnet address as is	No

Object	Attribute	Description	Mandatory
irb-instance	virtual-gateway-v4-address	Virtual Gateway IPv4 address	No
	virtual-gateway-v6-address	Virtual Gateway IPv6 address	No
	vlan-id [vlan-id]	IEEE 802.1q VLAN identifier for VLAN (1..4094)	Yes
	no-dhcp-relay	Exclude this irb from dhcp-relay	No
	apply-config-groups	Apply groups to the generated irb configuration	No
	vrf-instance [instance-id]	VRF instance identifier (1..255)NOTE: default is 1	No
server	mac-vrf-instance [instance-id]	MAC-VRF instance identifier (1..255)NOTE: default is 1	No
	apply-config-groups	Apply groups to the generated irb configuration	No
	server-name	Server name ; each server must be assigned a server-name	Yes
server	esi-lag-id	Assign an identifier for each server (1..1024)NOTE: Either esi-lag-id or single-home-id is mandatory	See Note
	single-home-id	Assign an identifier for a single home server (1..1024)NOTE: Either esi-lag-id or single-home-id is mandatory	See Note
	enable-pxe-boot	Enable PXE boot on this server	No

Object	Attribute	Description	Mandatory
	vlan-id-list [vlan-id-list]	List of VLANs for PEER to SERVER NOTE: With this config all VLANs will be in mac-vrf-instance NOTE: Either vlan-id-list or mac-vrf-instance vlan-id-list is mandatory	See Note
	interface [interface-name]	Interface or list of peer-to-server interfaces	Yes
	mac-vrf-instance [instance-id] vlan-id-list	List of VLANs for PEER to SERVER for mac-vrf-instance NOTE: Either vlan-id-list or mac-vrf-instance vlan-id-list is mandatory	See Note
	apply-config-groups	Apply groups to the generated interface configuration	No
	no-lacp	Do not configure LACP for this server	No

Table 2: Default and Auto-Derived Parameters

Attribute	Description	Default ValueAuto-Derived Value
[ae-device-count]	number of AE devices in the system	255 or platform specific
[ae-peer-to-peer]	PEER to PEER aggregate interface name	ae0
[local-address]	PEER to PEER address for ae0 ; derived from [peer-subnet] parameter	Example:[peer-subnet] == 172.16.12.0/31if peer_id == 1:local_address = 172.16.12.0/31else:local_address = 172.16.12.1/31
[ae-peer-to-server]	PEER to SERVER aggregate interface name	for dual-homed server:ae[esi-lag-id]for single-homed server:ae[1024 + esi-lag-id]
[system-id]	LACP system id	[system-id] + [esi-lag-id]

Attribute	Description	Default ValueAuto-Derived Value
[loopback-address]	Loopback addressAlso, the router-id and the local-address of overlay BGP session	[loopback-subnet]Example:[loopback-subnet]:192.0.2.10/32[loopback-address]:192.0.2.10/32
[router-id]	Router identifier	[loopback-address]
[virtual-gateway-v4-mac]	Virtual gateway mac for v4	Default is 00:00:5e:00:01:01
[virtual-gateway-v6-mac]	Virtual gateway mac for v6	Default is 00:00:5e:00:02:01
[virtual-gateway-address]	Virtual gateway addressNOTE: this can be specified by the:set services evpn evpn-vxlan irb irb_10subnet-address inet [ipv4-subnet-address] virtual-gateway-v4-address [virtual-gateway-v4-address]set services evpn evpn-vxlan irb irb_10subnet-address inet6 [ipv6-subnet-address] virtual-gateway-v6-address [virtual-gateway-v6-address]	Rightmost digit of [subnet-address] set to 254For exampleIf [subnet-address] == 10.0.1.11/24:[virtual-gateway-address] = 10.0.1.254
[policy-statement]	Policy to load-balance “per-packet”, aka per flowset policy-options policy-statement pplb then load-balance per-packetset routing-options forwarding-table export pplb	pplb
[neighbor-address]	lo0 address of neighbor PEER	[loopback-subnet]+-1 for the leftmost IPv4 address digitFor example, [loopback-subnet] ==192.0.2.10/32If [peer-id] == 1:[neighbor-address] =193.0.2.10else[neighbor-address] =191.0.2.10
[underlay-interface]	Interface for OSPF	ae0.0
[routing-instance]	Routing instance name	__SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_[m vrf-instance]
[l3-interface]	Routing interface name	irb.[vlan-id]
[vlan-name]	Bridge domain name	SERVICES_EVPN_EVPN_VXLAN_VLAN_[vlan-id]
[vni]	VXLAN identifier	10000 + [vlan-id]

Attribute	Description	Default ValueAuto-Derived Value
[ae-peer-to-server-subunit]	PEER to SERVER aggregate interface subunit	ae[server-id].0
[route-distinguisher]	Route Distinguisher	[loopback-address]:[mac-vrf-instance]Example:[route-distinguisher]:192.0.2.10:1
[vrf-target]	VRF Target Extended Community	target:1:[mac-vrf-instance]
[ae-unit]	Unit for aggregated interface (server)	ae[esi-lag-id].[mac-vrf-instance]Example:ae1.1
[as_number]	Autonomous system number	65000 + [peer-id]
[local-as]	Local as number for underlay	if peer_id == 1:local_as = 65000 + 4else:local_as = 65000 + 3
[local-address]	Local address for underlay	[peer-subnet]+-1Example for [peer-subnet] = 172.16.12.0/31if peer_id == 1:local_address = 172.16.12.0else:local_address = 172.16.12.1
[neighbor]	Neighbor address for underlay	[peer-subnet]+-1Example for [peer-subnet] = 172.16.12.0/31if peer_id == 1:neighbor = 172.16.12.1else:neighbor = 172.16.12.0
[peer-as]	Peer as number for underlay	if peer_id == 1:peer_as = 65000 + 3else:peer_as = 65000 + 4
[local-address]	Local address for overlay	[loopback-address]
[neighbor]	Neighbor address for overlay	[loopback-subnet]+-1 for the leftmost IPv4 address digitFor example, [loopback-subnet] ==192.0.2.10/32if [peer-id] == 1:[neighbor-address] =193.0.2.10else[neighbor-address] =191.0.2.10
[peer-as]	Peer as number for overlay	if peer_id == 1:peer_as = 65000 + peer_id + 1else:peer_as = 65000 + peer_id - 1

In the setup we used for this post, these are the configured attributes:

	Leaf-1	Leaf-2
peer-id	1	2
loopback-subnet	111.1.1.10/32	112.1.1.10/32
system-id	00:14:15:16:17:10	00:14:15:16:17:10
peer-subnet	172.16.12.0/31	172.16.12.0/31
interface-name	xe-0/0/46	xe-0/0/46
irb instance interface-name	IRB_100	IRB_100
irb instance subnet-address	10.0.1.11/24	10.0.1.10/24
irb instancevlan-id	100	100
irb instance interface-name	IRB_200	IRB_200
irb instance subnet-address	10.0.2.11/24	10.0.2.10/24
irb instancevlan-id	200	200
server-name	SERVER_1	SERVER_1
server esi-lag-id	1	1
server vlan-id-list	100	100
interface-name	xe-0/0/45	xe-0/0/45
server-name	SERVER_2	SERVER_2
server esi-lag-id	2	2
irb instancevlan-id	200	200
interface-name	xe-0/0/44	xe-0/0/44

These are the auto derived attributes:

	Leaf-1	Leaf-2
ae-peer-to-peer	ae0	ae0
Local-address	172.16.12.0/31	172.16.12.1/31
ae-peer-to-server	ae1	ae1
LACP system-id	00:14:15:16:17:11	00:14:15:16:17:11
ae-unit	ae1.1	ae1.1
vlan members	100	100
VNI	10100	10100
l3-interface	lrb.100	lrb.100
virtual-gateway-address	10.0.1.254	10.0.1.254
ae-peer-to-server	ae2	ae2

	Leaf-1	Leaf-2
LACP system-id	00:14:15:16:17:12	00:14:15:16:17:12
ae-unit	ae2.1	ae2.1
vlan members	200	200
VNI	10200	10200
l3-interface	lrb.200	lrb.200
virtual-gateway-address	10.0.2.254	10.0.2.254
MAC-VRF instance name	__SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1	__SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1
Loopback address	111.1.1.10/32	112.1.1.10/32
router-id	111.1.1.10	112.1.1.10
route-distinguisher	111.1.1.10:1	112.1.1.10:1
vrf-target	target:1:1	target:1:1
autonomous-system	65001	65002
Underlay BGP local-address	172.16.12.0	172.16.12.1
Underlay BGP local-as	65004	65003
Underlay BGP neighbor	172.16.12.1	172.16.12.0
Underlay peer-as	65003	65004
Overlay BGP local-address	111.1.1.10	112.1.1.10
Overlay BGP neighbor	112.1.1.10	111.1.1.10
Overlay BGP peer-as	65002	65001

1.4 Basic EZ-LAG Configuration

1.4.1 Peer 1 Configuration

Peer 1 Base Configuration

```

1 set version 23.2R1.13
2 set system host-name qfx1
3 set system root-authentication encrypted-password "[secret data]"
4 set system services ssh root-login allow
5 set system syslog user * any emergency
6 set system syslog file interactive-commands interactive-commands any
7 set system syslog file messages any notice
8 set system syslog file messages authorization info
9 set interfaces em0 unit 0 family inet address 192.168.1.101/24
10 set forwarding-options evpn-vxlan shared-tunnels
11 set routing-options static route 0.0.0.0/0 next-hop 192.168.1.1
12 set protocols lldp port-id-subtype interface-name
13 set protocols lldp interface all

```

Peer 1 EZ-LAG Configuration

```

1 set system scripts commit allow-transients
2 set system scripts commit file services_evpn_commit_script.py
3 set services evpn device-attribute peer-id 1

```

```

4 set services evpn device-attribute loopback-subnet 111.1.1.10/32
5 set services evpn device-attribute system-id 00:14:15:16:17:11
6 set services evpn device-attribute peer-to-peer peer-subnet inet 172.16.12.0/31
7 set services evpn device-attribute peer-to-peer peer-subnet interface-name xe-0/0/46
8 set services evpn evpn-vxlan irb IRB_100 vlan-id 100
9 set services evpn evpn-vxlan irb IRB_100 subnet-address inet 10.0.1.11/24
10 set services evpn evpn-vxlan irb IRB_200 vlan-id 200
11 set services evpn evpn-vxlan irb IRB_200 subnet-address inet 10.0.2.11/24
12 set services evpn evpn-vxlan server SERVER_1 esi-lag-id 1
13 set services evpn evpn-vxlan server SERVER_1 vlan-id-list 100
14 set services evpn evpn-vxlan server SERVER_1 interface xe-0/0/45
15 set services evpn evpn-vxlan server SERVER_2 esi-lag-id 2
16 set services evpn evpn-vxlan server SERVER_2 vlan-id-list 200
17 set services evpn evpn-vxlan server SERVER_2 interface xe-0/0/44

```

1.4.2 Peer 2 Configuration

Peer 2 Base Configuration

```

1 set version 23.2R1.13
2 set system host-name qfx2
3 set system root-authentication encrypted-password "[secret data]"
4 set system services ssh root-login allow
5 set system syslog user * any emergency
6 set system syslog file interactive-commands interactive-commands any
7 set system syslog file messages any notice
8 set system syslog file messages authorization info
9 set interfaces em0 unit 0 family inet address 192.168.1.102/24
10 set forwarding-options evpn-vxlan shared-tunnels
11 set routing-options static route 0.0.0.0/0 next-hop 192.168.1.1
12 set protocols lldp port-id-subtype interface-name
13 set protocols lldp interface all

```

Peer 2 EZ-LAG Configuration

```

1 set system scripts commit allow-transients
2 set system scripts commit file services_evpn_commit_script.py
3 set services evpn device-attribute peer-id 7
4 set services evpn device-attribute loopback-subnet 111.1.1.10/32
5 set services evpn device-attribute system-id 00:14:15:16:17:10
6 set services evpn device-attribute peer-to-peer peer-subnet inet 172.16.12.0/31
7 set services evpn device-attribute peer-to-peer peer-subnet interface-name xe-0/0/46
8 set services evpn evpn-vxlan irb IRB_100 vlan-id 100
9 set services evpn evpn-vxlan irb IRB_100 subnet-address inet 10.0.1.10/24
10 set services evpn evpn-vxlan irb IRB_200 vlan-id 200
11 set services evpn evpn-vxlan irb IRB_200 subnet-address inet 10.0.2.10/24
12 set services evpn evpn-vxlan server SERVER_1 esi-lag-id 1
13 set services evpn evpn-vxlan server SERVER_1 vlan-id-list 100
14 set services evpn evpn-vxlan server SERVER_1 vlan-id-list 200
15 set services evpn evpn-vxlan server SERVER_1 interface xe-0/0/45
16 set services evpn evpn-vxlan server SERVER_2 esi-lag-id 2
17 set services evpn evpn-vxlan server SERVER_2 vlan-id-list 100
18 set services evpn evpn-vxlan server SERVER_2 vlan-id-list 200
19 set services evpn evpn-vxlan server SERVER_2 interface xe-0/0/44

```

1.4.3 Host-1 Configuration Files (Centos 7)

/etc/sysconfig/network-scripts/ifcfg-ens2f0

```

1 NAME=ens2f0
2 UUID=4f8772aa-1c36-44d8-a1d5-8da3d7921eec
3 DEVICE=ens2f0
4 TYPE=Ethernet
5 BOOTPROTO=None
6 ONBOOT=yes
7 NM_CONTROLLED=no
8 SLAVE=yes
9 MASTER=bond0

```

/etc/sysconfig/network-scripts/ifcfg-ens2f1

```

1 NAME=ens2f1
2 DEVICE=ens2f1
3 TYPE=Ethernet
4 UUID=9b89bf2d-2d26-49e8-b1d1-c532816e1dcb
5 BOOTPROTO=none
6 ONBOOT=no
7 NM_CONTROLLED=no
8 SLAVE=yes
9 MASTER=bond0

```

/etc/sysconfig/network-scripts/ifcfg-bond0

```

1 DEVICE=bond0
2 TYPE=Bond
3 ONBOOT=yes
4 BONDING_OPTS="mode=802.3ad miimon=100 lacp_rate=fast"
5 NM_CONTROLLED=no
6 MTU="9000"

```

/etc/sysconfig/network-scripts/ifcfg-bond0.100

```

1 DEVICE=bond0.100
2 ONPARENT=yes
3 BOOTPROTO=none
4 DEFROUTE=yes
5 IPADDR=10.0.1.111
6 PREFIX=24
7 VLAN=yes
8 NM_CONTROLLED=no
9 ZONE=trusted

```

/etc/sysconfig/network-scripts/route-bond0.100

```

1 10.0.2.0/24 via 10.0.1.254
2 10.0.3.0/24 via 10.0.1.254
3 10.0.4.0/24 via 10.0.1.254

```

1.4.4 Host-2 Configuration (Ubuntu 20.04)

```

1 network:
2   version: 2
3   renderer: networkd
4   ethernets:
5     ens2f0:
6       match:
7         macaddress: a0:42:3f:3a:f7:76
8         set-name: ens2f0
9     ens2f1:
10      match:
11        macaddress: a0:42:3f:3a:f7:77
12        set-name: ens2f1
13   bonds:
14     bond0:
15       dhcp4: false
16       dhcp6: false
17       interfaces: [ ens2f0, ens2f1 ]
18       parameters:
19         mode: 802.3ad
20         mii-monitor-interval: 100
21   vlans:
22     bond0.200:
23       id: 200
24       link: bond0
25       addresses : [10.0.2.2/24]
26       routes:
27         - to: 10.0.1.0/24
28           via: 10.0.2.1
29         - to: 10.0.3.0/24
30           via: 10.0.2.1
31         - to: 10.0.4.0/24
32           via: 10.0.2.1

```

1.5 Verifications

Let's now look at the details of the configuration that has been generated by the commit script and verify that the setup is working as expected.

As explained at the beginning of this post, since the whole objective of this feature is to not expose the complexity of EVPN-VXLAN configuration to the user, we chose to implement it by using a commit script. The curious user can refer to the document listed in the "Useful Links" section to learn more about how commit scripts work on a Junos device. For the purpose of this post, we can simply say that a commit script is a tool that allows the user to verify at commit time that the configuration complies with certain user defined rules, and generate warnings, logs, or even change the configuration itself. It is the latter option that is used to implement EZ-LAG feature. In this case, when the user commits the configuration above, the commit script parses "services evpn" section of the configuration and generates other configuration sections that are not exposed by default in the configuration displayed with "show configuration" command. However, the user can always use "display commit-scripts" to see the entire configuration, the original one and the commit-script generated one, all merged into one configuration, with certain markers for easier reading.

The excerpt below shows the output of this special "show" command on Leaf-1 after committing the configuration listed above.

We can see that the commit script auto generated CLI configuration commands under interfaces, forwarding-options, policy-options, routing-instances, routing-options, and protocols stanzas.

```

1 {master:0}
2 root@qfx1] show configuration | display commit-scripts | display set
3 set version 23.2R1.13
4 set system host-name qfx1
5 set system root-authentication encrypted-password "$6$8xY0iVur$w8.f59mL/
   q7w4Vt04KZCCSape0vDokgktMxfJ2Z.MY.1pBjvVR5wPlhYiG.3sg31UsxjHnnWjHezBJ6kAQYnd0"
6 set system scripts commit allow-transients
7 set system scripts commit file services-evpn_commit_script.py
8 set system services ssh root-login allow
9 set system syslog user * any emergency
10 set system syslog file interactive-commands interactive-commands any
11 set system syslog file messages any notice
12 set system syslog file messages authorization info
13 set chassis aggregated-devices ethernet device-count 255
14 set chassis network-services enhanced-ip
15 set services evpn device-attribute peer-id 1
16 set services evpn device-attribute loopback-subnet 111.1.1.10/32
17 set services evpn device-attribute system-id 00:01:02:03:04:05
18 set services evpn device-attribute peer-to-peer peer-subnet inet 172.16.12.0/31
19 set services evpn device-attribute peer-to-peer peer-subnet interface-name xe-0/0/46
20 set services evpn evpn-vxlan irb IRB_100 vlan-id 100
21 set services evpn evpn-vxlan irb IRB_100 subnet-address inet 10.0.1.11/24
22 set services evpn evpn-vxlan irb IRB_200 vlan-id 200
23 set services evpn evpn-vxlan irb IRB_200 subnet-address inet 10.0.2.11/24
24 set services evpn evpn-vxlan server SERVER_1 esi-lag-id 1
25 set services evpn evpn-vxlan server SERVER_1 vlan-id-list 100
26 set services evpn evpn-vxlan server SERVER_1 interface xe-0/0/45
27 set services evpn evpn-vxlan server SERVER_2 esi-lag-id 2
28 set services evpn evpn-vxlan server SERVER_2 vlan-id-list 200
29 set services evpn evpn-vxlan server SERVER_2 interface xe-0/0/44
30 set interfaces xe-0/0/44 ether-options ethernet-switch-profile storm-control
   __SERVICES_EVPN_EVPN_VXLAN_STORM_CONTROL
31 set interfaces xe-0/0/44 ether-options 802.3ad ae2
32 set interfaces xe-0/0/45 ether-options ethernet-switch-profile storm-control
   __SERVICES_EVPN_EVPN_VXLAN_STORM_CONTROL
33 set interfaces xe-0/0/45 ether-options 802.3ad ae1
34 set interfaces xe-0/0/46 ether-options 802.3ad ae0
35 set interfaces ae0 aggregated-ether-options lacp active
36 set interfaces ae0 unit 0 family inet address 172.16.12.0/31
37 set interfaces ae1 vlan-tagging
38 set interfaces ae1 encapsulation flexible-ethernet-services
39 set interfaces ae1 esi auto-derive type-1-lacp
40 set interfaces ae1 esi all-active
41 set interfaces ae1 aggregated-ether-options lacp active

```

```

42 set interfaces ae1 aggregated-ether-options lacp system-id 00:01:02:03:04:06
43 set interfaces ae1 unit 1 family ethernet-switching interface-mode trunk
44 set interfaces ae1 unit 1 family ethernet-switching vlan members 100
45 set interfaces ae2 vlan-tagging
46 set interfaces ae2 encapsulation flexible-ethernet-services
47 set interfaces ae2 esi auto-derive type-1-lacp
48 set interfaces ae2 esi all-active
49 set interfaces ae2 aggregated-ether-options lacp active
50 set interfaces ae2 aggregated-ether-options lacp system-id 00:01:02:03:04:07
51 set interfaces ae2 unit 1 family ethernet-switching interface-mode trunk
52 set interfaces ae2 unit 1 family ethernet-switching vlan members 200
53 set interfaces em0 unit 0 family inet address 10.87.11.81/25
54 set interfaces irb unit 100 virtual-gateway-accept-data
55 set interfaces irb unit 100 family inet address 10.0.1.11/24 virtual-gateway-address 10.0.1.254
56 set interfaces irb unit 100 virtual-gateway-v4-mac 00:00:5e:00:01:01
57 set interfaces irb unit 200 virtual-gateway-accept-data
58 set interfaces irb unit 200 family inet address 10.0.2.11/24 virtual-gateway-address 10.0.2.254
59 set interfaces irb unit 200 virtual-gateway-v4-mac 00:00:5e:00:01:01
60 set interfaces lo0 unit 0 family inet address 111.1.1.10/32 primary
61 set interfaces lo0 unit 0 family inet address 111.1.1.10/32 preferred
62 set forwarding-options storm-control-profiles __SERVICES_EVPN_EVPN_VXLAN_STORM_CONTROL all
    bandwidth-percentage 1
63 set forwarding-options evpn-vxlan shared-tunnels
64 set forwarding-options vxlan-routing next-hop 45056
65 set forwarding-options vxlan-routing interface-num 8192
66 set forwarding-options vxlan-routing overlay-ecmp
67 set policy-options policy-statement EXPORT-L00 term LOOPBACK from interface lo0.0
68 set policy-options policy-statement EXPORT-L00 term LOOPBACK then accept
69 set policy-options policy-statement EXPORT-L00 term REJECT then reject
70 set policy-options policy-statement p1b then load-balance per-packet
71 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 instance-type mac-vrf
72 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 protocols evpn encapsulation vxlan
73 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 protocols evpn default-gateway no-
    gateway-community
74 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 vtep-source-interface lo0.0
75 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 service-type vlan-aware
76 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 interface ae1.1
77 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 interface ae2.1
78 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 route-distinguisher 111.1.1.10:1
79 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 vrf-target target:1:1
80 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 vlans
    SERVICES_EVPN_EVPN_VXLAN_VLAN_100 vlan-id 100
81 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 vlans
    SERVICES_EVPN_EVPN_VXLAN_VLAN_100 l3-interface irb.100
82 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 vlans
    SERVICES_EVPN_EVPN_VXLAN_VLAN_100 vxlan vni 10100
83 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 vlans
    SERVICES_EVPN_EVPN_VXLAN_VLAN_200 vlan-id 200
84 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 vlans
    SERVICES_EVPN_EVPN_VXLAN_VLAN_200 l3-interface irb.200
85 set routing-instances __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1 vlans
    SERVICES_EVPN_EVPN_VXLAN_VLAN_200 vxlan vni 10200
86 set routing-options router-id 111.1.1.10
87 set routing-options autonomous-system 65001
88 set routing-options static route 0.0.0.0/0 next-hop 10.87.11.126
89 set routing-options forwarding-table export p1b
90 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_UNDERLAY type external
91 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_UNDERLAY local-address 172.16.12.0
92 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_UNDERLAY export EXPORT-L00
93 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_UNDERLAY local-as 65004
94 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_UNDERLAY multipath multiple-as
95 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_UNDERLAY neighbor 172.16.12.1 peer-as
    65003
96 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_OVERERLAY type external
97 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_OVERERLAY multihop ttl 2
98 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_OVERERLAY multihop no-nexthop-change
99 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_OVERERLAY local-address 111.1.1.10
100 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_OVERERLAY family evpn signaling
101 set protocols bgp group __SERVICES_EVPN_EVPN_VXLAN_EBGP_OVERERLAY neighbor 112.1.1.10 peer-as
    65002
102 set protocols evpn no-core-isolation
103 set protocols lldp port-id-subtype interface-name
104 set protocols lldp interface all
105 set protocols loop-detect enhanced interface ae1 vlan-id 100
106 set protocols loop-detect enhanced interface ae1 loop-detect-action interface-down
107 set protocols loop-detect enhanced interface ae1 transmit-interval 1s
108 set protocols loop-detect enhanced interface ae1 revert-interval 60
109 set protocols loop-detect enhanced interface ae2 vlan-id 200
110 set protocols loop-detect enhanced interface ae2 loop-detect-action interface-down
111 set protocols loop-detect enhanced interface ae2 transmit-interval 1s
112 set protocols loop-detect enhanced interface ae2 revert-interval 60

```

You see the benefits of EZ-LAG feature by comparing the length of the original config to that of the auto-generated one and understand the time saving to configure it and having less chances to make configuration mistakes at the same time.

Notice the highlighted system-ids for both ae interfaces that are autogenerated and must match on the two peers.

Let's now verify that the setup is working as expected by running some operational CLI commands.

Leaf-1

```

1 {master:0}
2 root@qfx1] show interfaces terse
3 Interface Admin Link Proto Local Remote
4
5 [snip]
6
7 xe-0/0/44 up up
8 xe-0/0/44.1 up up aenet --] ae2.1
9 xe-0/0/44.32767 up up aenet --] ae2.32767
10 xe-0/0/45 up up
11 xe-0/0/45.1 up up aenet --] ae1.1
12 xe-0/0/45.32767 up up aenet --] ae1.32767
13 xe-0/0/46 up up
14 xe-0/0/46.0 up up aenet --] ae0.0
15
16 [snip]
17
18 ae0 up up
19 ae0.0 up up inet 172.16.12.0/31
20 ae1 up up
21 ae1.1 up up eth-switch
22 ae1.32767 up up
23 ae2 up up
24 ae2.1 up up eth-switch
25 ae2.32767 up up
26
27 [snip]
28
29 irb up up
30 irb.100 up up inet 10.0.1.11/24
31                                     10.0.1.254/24
32 irb.200 up up inet 10.0.2.11/24
33                                     10.0.2.254/24
34
35 [snip]
36
37 lo0.0 up up inet 111.1.1.10 --] 0/0
38 lo0.16384 up up inet 127.0.0.1 --] 0/0
39 lo0.16385 up up inet
40
41 [snip]
```

Leaf-2

```

1 {master:0}
2 root@qfx2] show interfaces terse
3 Interface Admin Link Proto Local Remote
4 gr-0/0/0 up up
5 pfe-0/0/0 up up
6 pfe-0/0/0.16383 up up inet
7                                     inet6
8 pfh-0/0/0 up up
9 pfh-0/0/0.16383 up up inet
10 pfh-0/0/0.16384 up up inet
11 xe-0/0/44 up up
12 xe-0/0/44.1 up up aenet --] ae2.1
13 xe-0/0/44.32767 up up aenet --] ae2.32767
14 xe-0/0/45 up up
15 xe-0/0/45.1 up up aenet --] ae1.1
16 xe-0/0/45.32767 up up aenet --] ae1.32767
17 xe-0/0/46 up up
18 xe-0/0/46.0 up up aenet --] ae0.0
```

```

19 et-0/0/48 up up
20 et-0/0/48.16386 up up
21 et-0/0/50 up up
22 et-0/0/50.16386 up up
23 ae0 up up
24 ae0.0 up up inet 172.16.12.1/31
25 ae1 up up
26 ae1.1 up up eth-switch
27 ae1.32767 up up
28 ae2 up up
29 ae2.1 up up eth-switch
30 ae2.32767 up up
31 bme0 up up
32 bme0.0 up up inet 128.0.0.1/2
33                                     128.0.0.4/2
34                                     128.0.0.63/2
35 cbp0 up up
36 dsc up up
37 em0 up up
38 em0.0 up up inet 10.87.11.82/25
39 em1 up down
40 em2 up up
41 em2.32768 up up inet 192.168.1.2/24
42 em3 up up
43 esi up up
44 fti0 up up
45 gre up up
46 ipip up up
47 irb up up
48 irb.100 up up inet 10.0.1.11/24
49                                     10.0.1.254/24
50 irb.200 up up inet 10.0.2.11/24
51                                     10.0.2.254/24
52 jsrv up up
53 jsrv.1 up up inet 128.0.0.127/2
54 lo0 up up
55 lo0.0 up up inet 112.1.1.10 --] 0/0
56 lo0.16384 up up inet 127.0.0.1 --] 0/0
57 lo0.16385 up up inet
58 lsi up up
59 mtun up up
60 pimd up up
61 pime up up
62 pip0 up up
63 tap up up
64 vme up down
65 vtep up up
66 vtep.32768 up up
67 vtep.32769 up up eth-switch
68 vtep.32771 up up
69
70 {master:0}
71 root@qfx2]

```

The above excerpt shows that the script created interfaces ae0.0, ae1.1, ae2.1, irb.100, irb.200, and lo0.0 and assigned the proper addresses.

It's also important to verify that ae1 and ae2 interfaces are “up up” on both leafs, because ESI is configured in all-active mode.

Let's first look at LACP status and statistics. It is important that Host-1 sees Leaf-1 and Leaf-2 as the same LACP system, therefore it does not know that it is dual-homed to two separate leafs. This is verified below because the System ID is identical for the “Actor” role on both leafs.

Leaf-1

```

1 {master:0}
2 root@qfx1] show lacp interfaces ae1 extensive
3 Aggregated interface: ae1
4   LACP state: Role Exp Def Dist Col Syn Aggr Timeout Activity
5     xe-0/0/45 Actor No No Yes Yes Yes Yes Fast Active
6     xe-0/0/45 Partner No No Yes Yes Yes Yes Fast Active
7   LACP protocol: Receive State Transmit State Mux State
8     xe-0/0/45 Current Fast periodic Collecting distributing
9   LACP info: Role System System Port Port Port

```

```

10          priority identifier priority number key
11      xe-0/0/45 Actor 127 00:01:02:03:04:06 127 2 2
12      xe-0/0/45 Partner 65535 90:e2:ba:d9:30:64 255 1 15
13
14 {master:0}
15 root@qfx1]

```

Leaf-2

```

1 {master:0}
2 root@qfx2] show lacp interfaces ae1 extensive
3 Aggregated interface: ae1
4   LACP state: Role Exp Def Dist Col Syn Aggr Timeout Activity
5     xe-0/0/45 Actor No No Yes Yes Yes Yes Fast Active
6     xe-0/0/45 Partner No No Yes Yes Yes Yes Fast Active
7   LACP protocol: Receive State Transmit State Mux State
8     xe-0/0/45 Current Fast periodic Collecting distributing
9   LACP info: Role System System Port Port Port
10          priority identifier priority number key
11      xe-0/0/45 Actor 127 00:01:02:03:04:06 127 2 2
12      xe-0/0/45 Partner 65535 90:e2:ba:d9:30:64 255 2 15
13
14 {master:0}
15 root@qfx2]

```

Let's now have a closer look at one of the AE interfaces.

Leaf-1

```

1 {master:0}
2 root@qfx1] show interfaces ae1 extensive
3 Physical interface: ae1, Enabled, Physical link is Up
4   Interface index: 692, SNMP ifIndex: 522, Generation: 183
5   Link-level type: Flexible-Ethernet, MTU: 1518, Speed: 10Gbps, BPDU Error: None, Ethernet-
6     Switching Error: None, MAC-REWRITE Error: None, Loopback: Disabled, Source filtering:
7     Disabled,
8   Flow control: Disabled, Minimum links needed: 1, Minimum bandwidth needed: 1bps
9   Device flags : Present Running
10  Interface flags: SNMP-Traps Internal: 0x4000
11  Current address: 80:7f:f8:d2:d0:29, Hardware address: 80:7f:f8:d2:d0:29
12  Ethernet segment value: 01:90:e2:ba:d9:30:64:00:0f:00, Mode: all-active
13  Last flapped : 2023-08-28 09:28:48 UTC (02:31:09 ago)
14  Statistics last cleared: Never
15  Traffic statistics:
16    Input bytes : 1297388 1256 bps
17    Output bytes : 1332988 1256 bps
18    Input packets: 9983 1 pps
19    Output packets: 9957 1 pps
20  IPv6 transit statistics:
21    Input bytes : 0
22    Output bytes : 0
23    Input packets: 0
24    Output packets: 0
25  Input errors:
26    Errors: 0, Drops: 0, Framing errors: 0, Runts: 0, Giants: 0, Policed discards: 0, Resource
27    errors: 0
28  Output errors:
29    Carrier transitions: 1, Errors: 0, Drops: 0, MTU errors: 0, Resource errors: 0
30  Egress queues: 10 supported, 5 in use
31  Queue counters: Queued packets Transmitted packets Dropped packets
32    0 566 566 0
33    3 0 0 0
34    4 0 0 0
35    7 9386 9386 0
36    8 9 9 0
37  Queue number: Mapped forwarding classes
38    0 best-effort
39    3 fcoe
40    4 no-loss
41    7 network-control
42    8 mcast
43
44  Logical interface ae1.1 (Index 878) (SNMP ifIndex 537) (HW Token 1) (Generation 339)
45  Flags: Up SNMP-Traps 0x20024000 Encapsulation: Ethernet-Bridge
46  Statistics Packets pps Bytes bps
47  Bundle:
48    Input : 0 0 0 0
49    Output: 3 0 138 0

```

```

47 Adaptive Statistics:
48   Adaptive Adjusts: 0
49   Adaptive Scans : 0
50   Adaptive Updates: 0
51 Link:
52   xe-0/0/45.1
53   Input : 0 0 0 0
54   Output: 3 0 138 0
55
56 Aggregate member links: 1
57
58 Marker Statistics: Marker Rx Resp Tx Unknown Rx Illegal Rx
59   xe-0/0/45.1 0 0 0 0
60 Protocol eth-switch, MTU: 1518, Generation: 305, Route table: 8, Mesh Group: __all_ces__,
   EVPN multi-homed status: Forwarding, EVPN multi-homed ESI Split Horizon Label: 0, Next-
   hop: 1804,
61 vpls-status: up
62 Local Bias Logical Interface Name: vtep.32770, Index: 881, VXLAN Endpoint Address: 112.1.1.10
63 Flags: Trunk-Mode, 0x4000000
64
65 Logical interface ae1.32767 (Index 879) (SNMP ifIndex 529) (Generation 340)
66 Flags: Up SNMP-Traps 0x4004000 VLAN-Tag [ 0x0000.0 ] Encapsulation: ENET2
67 Statistics Packets pps Bytes bps
68 Bundle:
69   Input : 0 0 0 0
70   Output: 0 0 0 0
71 Adaptive Statistics:
72   Adaptive Adjusts: 0
73   Adaptive Scans : 0
74   Adaptive Updates: 0
75 Link:
76   xe-0/0/45.32767
77   Input : 0 0 0 0
78   Output: 329 0 134873 0
79 LACP info: Role System System Port Port Port
80             priority identifier priority number key
81   xe-0/0/45.32767 Actor 127 00:01:02:03:04:06 127 2 2
82   xe-0/0/45.32767 Partner 65535 90:e2:ba:d9:30:64 255 1 15
83 LACP Statistics: LACP Rx LACP Tx Unknown Rx Illegal Rx
84   xe-0/0/45.32767 9048 9007 0 0
85 Marker Statistics: Marker Rx Resp Tx Unknown Rx Illegal Rx
86   xe-0/0/45.32767 0 0 0 0
87
88 {master:0}
89 root@qfx1]

```

EZ-LAG sets LACP auto-derive mode to “type-1-lacp” and, in this mode, the ESI is the system ID and the administrative key on the remote CE device in the LACP link (partner). In our case, the remote CE device is Host-1

- 01 - ESI Type-1
- 90:e2:ba:d9:30:64 - MAC address of the bond on Host-1
- 00:0f - admin key on Host-1
- 00 - the last octet is always 0

These information can be easily verified on Host-1

```

1 [root@5a7s38 ~]# ifconfig bond0 | grep ether
2   ether 90:e2:ba:d9:30:64 txqueuelen 1000 (Ethernet)
3 [root@5a7s38 ~]#
4 [root@5a7s38 ~]# tcpdump -i ens2f0 -vvv
5 tcpdump: listening on ens2f0, link-type EN10MB (Ethernet), capture size 262144 bytes
6 19:53:42.884097 LACPV1, length 110
7   Actor Information TLV (0x01), length 20
8     System 90:e2:ba:d9:30:64 (oui Unknown), System Priority 65535, Key 15, Port 1, Port
       Priority 255
9     State Flags [Activity, Timeout, Aggregation, Synchronization, Collecting, Distributing]
10    0x0000: ffff 90e2 bad9 3064 000f 00ff 0001 3f00
11    0x0010: 0000
12   Partner Information TLV (0x02), length 20
13     System 00:01:02:03:04:06 (oui Unknown), System Priority 127, Key 2, Port 2, Port
       Priority 127
14     State Flags [Activity, Timeout, Aggregation, Synchronization, Collecting, Distributing]
15    0x0000: 007f 0001 0203 0406 0002 007f 0002 3f00

```

```

16      0x0010: 0000
17      Collector Information TLV (0x03), length 16
18      Max Delay 0
19      0x0000: 0000 0000 0000 0000 0000 0000 0000
20      Terminator TLV (0x00), length 0

```

Verify that ESI is identical on Leaf-2

```

1 {master:0}
2 root@qfx2] show interfaces ae1 extensive | match segment
3 Ethernet segment value: 01:90:e2:ba:d9:30:64:00:0f:00, Mode: all-active
4
5 {master:0}
6 root@qfx2]

```

It is important to verify that for each ae interface, only one leaf is forwarding BUM traffic to the server and the other leaf is blocking it.

```

1 {master:0}
2 root@qfx2] show interfaces ae1 extensive | match multi-homed
3 Protocol eth-switch, MTU: 1518, Generation: 301, Route table: 8, Mesh Group: __all_ces__,
4 EVPN multi-homed status: Blocking BUM Traffic to ESI, EVPN multi-homed ESI Split Horizon
5 Label: 0,
6 {master:0}
7 root@qfx2]

```

In our case, we can see that EVPN multi-homed status on Leaf-1 is “Forwarding” and that on Leaf-2, the status is “Blocking BUM traffic to ESI”.

From BGP perspective, we need to have two BGP sessions, one for the underlay and another for the overlay. Here both are “Established” and Leaf-1 received 12 EVPN routes.

```

1 {master:0}
2 root@qfx1] show bgp summary
3 Threading mode: BGP I/O
4 Default eBGP mode: advertise - accept, receive - accept
5 Groups: 2 Peers: 2 Down peers: 0
6 Table Tot Paths Act Paths Suppressed History Damp State Pending
7 bgp.evpn.0
8
9      12 12 0 0 0 0
10
11     1 1 0 0 0 0
12 Peer AS InPkt OutPkt OutQ Flaps Last Up/Dwn State|#Active/Received/Accepted/Damped...
13 112.1.1.10 65002 130 131 0 0 55:16 Establ
14   bgp.evpn.0: 12/12/12/0
15   __SERVICES_EVPN_EVPN_VXLAN_MAC_VRF_1.evpn.0: 12/12/12/0
16   __default_evpn__.evpn.0: 0/0/0/0
17 172.16.12.1 65003 127 125 0 0 55:19 Establ
18   inet.0: 1/1/1/0
19 {master:0}
20 root@qfx1]

```

Now, it’s important to check EVPN routes that have been exchanged between the two peers, after verifying reachability between Host-1 and Host-2.

In the context of this post, we’re more particularly interested in Type 1 EVPN routes that are used for per Ethernet segment auto-discovery to advertise EVPN multi-homing mode.

```

1 {master:0}
2 root@qfx1] show route table bgp.evpn.0 protocol bgp
3
4 bgp.evpn.0: 44 destinations, 44 routes (44 active, 0 holddown, 0 hidden)
5 + = Active Route, - = Last Active, * = Both
6
7 1:112.1.1.10:0::0190e2bad1e230000f00::FFFF:FFFF/192 AD/ESI
8      *[BGP/170] 02:25:52, localpref 100, from 112.1.1.10
9      AS path: 65002 I, validation-state: unverified
10      ] to 172.16.12.1 via ae0.0
11 1:112.1.1.10:0::0190e2bad93064000f00::FFFF:FFFF/192 AD/ESI

```

```

12      *[BGP/170] 02:25:53, localpref 100, from 112.1.1.10
13      AS path: 65002 I, validation-state: unverified
14      ] to 172.16.12.1 via ae0.0
15 1:112.1.1.10:0::050000fdea0000277400::FFFF:FFFF/192 AD/ESI
16      *[BGP/170] 04:09:41, localpref 100, from 112.1.1.10
17      AS path: 65002 I, validation-state: unverified
18      ] to 172.16.12.1 via ae0.0
19 1:112.1.1.10:0::050000fdea000027d800::FFFF:FFFF/192 AD/ESI
20      *[BGP/170] 04:09:41, localpref 100, from 112.1.1.10
21      AS path: 65002 I, validation-state: unverified
22      ] to 172.16.12.1 via ae0.0
23 1:112.1.1.10:1::0190e2bad1e230000f00::0/192 AD/EVI
24      *[BGP/170] 02:25:53, localpref 100, from 112.1.1.10
25      AS path: 65002 I, validation-state: unverified
26      ] to 172.16.12.1 via ae0.0
27 1:112.1.1.10:1::0190e2bad93064000f00::0/192 AD/EVI
28      *[BGP/170] 02:25:54, localpref 100, from 112.1.1.10
29      AS path: 65002 I, validation-state: unverified
30      ] to 172.16.12.1 via ae0.0
31 2:112.1.1.10:1::10100::00:00:5e:00:01:01/304 MAC/IP
32      *[BGP/170] 04:09:41, localpref 100, from 112.1.1.10
33      AS path: 65002 I, validation-state: unverified
34      ] to 172.16.12.1 via ae0.0, Push 631
35 2:112.1.1.10:1::10100::80:7f:f8:d2:02:60/304 MAC/IP
36      *[BGP/170] 04:09:41, localpref 100, from 112.1.1.10
37      AS path: 65002 I, validation-state: unverified
38      ] to 172.16.12.1 via ae0.0, Push 631
39 2:112.1.1.10:1::10100::90:e2:ba:d9:30:64/304 MAC/IP
40      *[BGP/170] 01:34:16, localpref 100, from 112.1.1.10
41      AS path: 65002 I, validation-state: unverified
42      ] to 172.16.12.1 via ae0.0, Push 631
43 2:112.1.1.10:1::10200::00:00:5e:00:01:01/304 MAC/IP
44      *[BGP/170] 04:09:41, localpref 100, from 112.1.1.10
45      AS path: 65002 I, validation-state: unverified
46      ] to 172.16.12.1 via ae0.0, Push 637
47 2:112.1.1.10:1::10200::80:7f:f8:d2:02:60/304 MAC/IP
48      *[BGP/170] 04:09:41, localpref 100, from 112.1.1.10
49      AS path: 65002 I, validation-state: unverified
50      ] to 172.16.12.1 via ae0.0, Push 637
51 2:112.1.1.10:1::10200::90:e2:ba:d1:e2:30/304 MAC/IP
52      *[BGP/170] 01:34:16, localpref 100, from 112.1.1.10
53      AS path: 65002 I, validation-state: unverified
54      ] to 172.16.12.1 via ae0.0, Push 637
55 2:112.1.1.10:1::10100::00:00:5e:00:01:01::10.0.1.254/304 MAC/IP
56      *[BGP/170] 04:09:41, localpref 100, from 112.1.1.10
57      AS path: 65002 I, validation-state: unverified
58      ] to 172.16.12.1 via ae0.0, Push 631
59 2:112.1.1.10:1::10100::80:7f:f8:d2:02:60::10.0.1.11/304 MAC/IP
60      *[BGP/170] 04:09:41, localpref 100, from 112.1.1.10
61      AS path: 65002 I, validation-state: unverified
62      ] to 172.16.12.1 via ae0.0, Push 631
63 2:112.1.1.10:1::10100::90:e2:ba:d9:30:64::10.0.1.111/304 MAC/IP
64      *[BGP/170] 00:19:07, localpref 100, from 112.1.1.10
65      AS path: 65002 I, validation-state: unverified
66      ] to 172.16.12.1 via ae0.0, Push 631
67 2:112.1.1.10:1::10200::00:00:5e:00:01:01::10.0.2.254/304 MAC/IP
68      *[BGP/170] 04:09:41, localpref 100, from 112.1.1.10
69      AS path: 65002 I, validation-state: unverified
70      ] to 172.16.12.1 via ae0.0, Push 637
71 2:112.1.1.10:1::10200::80:7f:f8:d2:02:60::10.0.2.11/304 MAC/IP
72      *[BGP/170] 04:09:41, localpref 100, from 112.1.1.10
73      AS path: 65002 I, validation-state: unverified
74      ] to 172.16.12.1 via ae0.0, Push 637
75 2:112.1.1.10:1::10200::90:e2:ba:d1:e2:30::10.0.2.112/304 MAC/IP
76      *[BGP/170] 00:19:07, localpref 100, from 112.1.1.10
77      AS path: 65002 I, validation-state: unverified
78      ] to 172.16.12.1 via ae0.0, Push 637
79 3:112.1.1.10:1::10100::112.1.1.10/248 IM
80      *[BGP/170] 04:09:42, localpref 100, from 112.1.1.10
81      AS path: 65002 I, validation-state: unverified
82      ] to 172.16.12.1 via ae0.0
83 3:112.1.1.10:1::10200::112.1.1.10/248 IM
84      *[BGP/170] 04:09:42, localpref 100, from 112.1.1.10
85      AS path: 65002 I, validation-state: unverified
86      ] to 172.16.12.1 via ae0.0
87 4:112.1.1.10:0::0190e2bad1e230000f00:112.1.1.10/296 ES
88      *[BGP/170] 02:25:53, localpref 100, from 112.1.1.10
89      AS path: 65002 I, validation-state: unverified
90      ] to 172.16.12.1 via ae0.0
91 4:112.1.1.10:0::0190e2bad93064000f00:112.1.1.10/296 ES
92      *[BGP/170] 02:25:54, localpref 100, from 112.1.1.10

```

```
93          AS path: 65002 I, validation-state: unverified
94      ] to 172.16.12.1 via ae0.0
95
96 {master:0}
97 root@qfx1]
```

In the CLI output above, we notice that there are two Type-1 EVPN routes per host, these are AD/ESI and AD/EVI routes. the AD/ESI route advertises the Ethernet segment to which the host is connected and the AD/EVI route advertises the remote EVPN to which the route belongs.

By looking at the ESIs in these routes and recalling what we explained above about how are these Type-1 ESI are constructed, we can identify that the 1st and 5th Type-1 routes are relative to Host-2, and that the 2nd and 6th Type-1 routes are relative to Host-1.

Now, how about the 3rd and 4th routes? Let's dissect the ESI of these routes and figure it out.

ESI "050000fdea0000277400" can be deconstructed as follows:

- 0x05 : Autonomous System Number (ASN) based ESI
- 0x0000fdea : ASN = 65002 (decimal)
- 0x00002774 : Local Discriminator = 10100 (decimal). In this case, the VNI is used as a Local Discriminator
- 0x00 : the last octet is always zero

This Type-1 route is in fact relative to the Virtual Gateway that was auto-derived from the EZ-LAG configuration. by default, EZ-LAG configures IRB interfaces with a virtual gateway, but there are CLI knobs available to change that into anycast mode. Those commands are:

```
1 set services evpn global-parameters anycast-mac [mac_address]
2 set services evpn evpn-vxlan irb [irb_instance] use-anycast-address
```

We verified by testing, that when this mode is used, the 3rd and 4th Type-1 routes above do not show up anymore.

1.6 Conclusion

In this post, we explained EZ-LAG feature in details. This new feature was introduced by Juniper as an alternative to MC-LAG and based on EVPN-VXLAN technology. We showed that the configuration of this feature has been made much simpler than ESI-LAG, which requires a full-fledged EVPN-VXLAN configuration. EZ-LAG hides the complexity of an ESI-LAG configuration while still providing the same feature and the same service. The knobs supported by EZ-LAG CLI allow the user to tailor the configuration and avoid conflicts with existing one for other features on the same switch.

1.7 Useful links

- Understanding Multi-Chassis Link Aggregation Groups: <https://www.juniper.net/documentation/us/en/software/junos/mc-lag/topics/concept/mc-lag-feature-summary-best-practices.html>
- Commit Scripts Overview: <https://www.juniper.net/documentation/us/en/software/junos/automation-scripting/topics/concept/junos-script-automation-commit-script-overview.html>

1.8 Glossary

- AD : Auto Discovery
- ASN : Autonomous System Number
- BGP : Border Gateway Protocol
- CLI : Command Line Interface
- ESI : Ethernet Segment Identifier
- EVI : Ethernet VPN Instance
- EVPN : Ethernet Virtual Private Network
- LAG : Link Aggregation Group, aka bond or port channel
- MC-LAG : Multi-Chassis LAG
- OSPF : Open Shortest Path First
- VNI : Virtual Network Identifier
- VXLAN : Extended Virtual Local Area Network

1.9 Acknowledgments

Many thanks to Aninda Chatterjee, Santhosh Somasekharan Nair and Selvakumar Sivaraj



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.

Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.

Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com

HPE

